



5η Εργαστηριακή Άσκηση

Ψηφιακά Συστήματα VLSI

Παναγιώτης Γερασιμόπουλος 03115208
personal@devcol.com

1. Deserializer

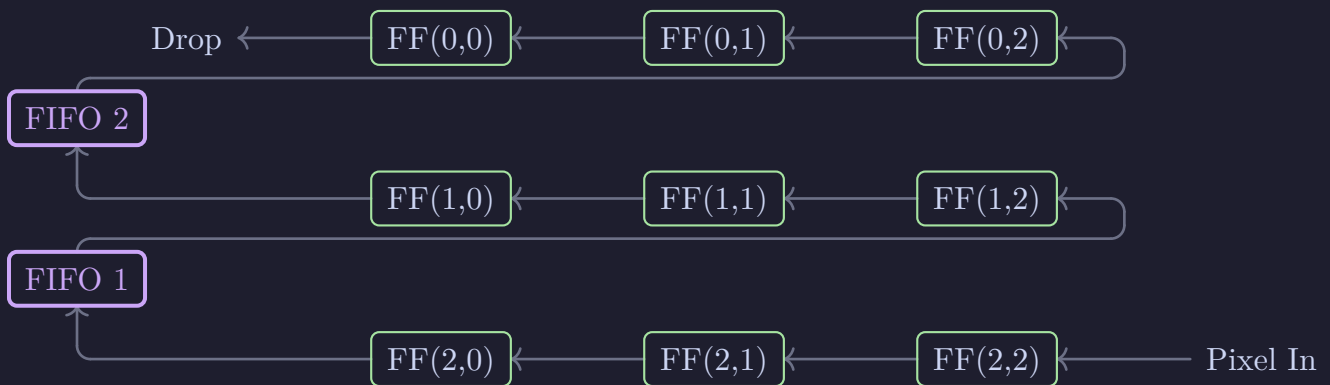
1.1. Fifo

Για το implementation της fifo θα φτιάξουμε έναν ring buffer χρησιμοποιώντας ένα κομμάτι block ram και έναν pointer που θα δείχνει μια θέση μετά από το τελευταίο στοιχείο που γράψαμε και θα επιστρέφει στο 0 όταν φτάσει το τέλος του buffer. Έτσι σε έναν κύκλο ο ring buffer θα δίνει στην έξοδο το στοιχείο που δείχνει ο pointer, θα γράφει την νέα τιμή στην θέση του και θα προχωράει στο επόμενο. Το κύκλωμα θα έχει σχετικά σταθερή κατανάλωση ισχύος αδιάφορα του βάθους της fifo και μικρό overhead υλικού.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 package buffer_pkg is
6     type ring_buffer_type is array (natural range <=>) of std_logic_vector(7 downto 0);
7 end package buffer_pkg;
8 -----
9 library ieee;
10 use ieee.std_logic_1164.all;
11 use ieee.numeric_std.all;
12 use work.buffer_pkg.all;
13
14 entity RingBuffer is
15     generic(
16         Depth : integer := 1024
17     );
18     port(
19         Clk, RstN, Enable : in std_logic;
20         DataIn             : in std_logic_vector(7 downto 0);
21         DataOut            : out std_logic_vector(7 downto 0)
22     );
23 end entity RingBuffer;
24
25 architecture Behavioral of RingBuffer is
26     signal RamBuffer : ring_buffer_type(0 to Depth - 1);
27     signal Pointer    : natural range 0 to Depth - 1;
28     -- NOTE(acol): magic to force BRam
29     attribute ram_style : string;
30     attribute ram_style of RamBuffer : signal is "block";
31 begin
32     process(Clk, RstN) begin
33         if RstN = '0' then
34             Pointer <= 0;
35             DataOut <= (others => '0');
36
37         elsif rising_edge(Clk) then
38             if Enable = '1' then
39                 DataOut <= RamBuffer(Pointer);
40                 RamBuffer(Pointer) <= DataIn;
41
42                 if Pointer = Depth - 1 then
43                     Pointer <= 0;
44                 else
45                     Pointer <= Pointer + 1;
46                 end if;
47             end if;
48         end if;
49     end process;
50 end architecture;
51
```

1.2. Window Generation

Για την δημιουργία του 3×3 παράθυρου θα χρησιμοποιήσουμε 2 fifo και ένα grid από flip flop. Η είσοδοι θα συνδεθούν απευθείας με το τελευταίο επίπεδο των flipflop τα οποία σε κάθε κύκλο θα κάνουν slide αριστερά. Οι έξοδοι του θα συνδεθούν με την είσοδο της πρώτης fifo της οποίας η έξοδος θα συνδέεται με την σειρά της στην είσοδο του ενδιαμέσου επιπέδου των flip flop. Τέλος, αυτό επαναλαμβάνετε άλλη μια φορά με το ενδιαμέσο επίπεδο των flipflop, την δεύτερη fifo και το πρώτο επίπεδο. Τα δεδομένα που βγαίνουν από το πρώτο επίπεδο τα αγνοούμε.



Με αυτόν τον τρόπο μπορούμε να χρησιμοποιήσουμε μόνο 2 fifo και αφού κρατάμε τα δεδομένα στο grid από flipflop αντί να συνδέσουμε τις fifo σε σειρά μπορούμε να μειώσουμε το βάθος τους κατά 3. Τέλος το routing από κάτω προς τα πάνω και δεξιά προς τα αριστερά θα κάνει το παράθυρο να είναι σε σειρά pixel της εικόνας με το κεντρικό να είναι το pixel που επεξεργαζόμαστε.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use ieee.numeric_std.all;
4 use work.all;
5
6 entity WindowGenerator is
7   generic(
8     ImageWidth : integer := 32
9   );
10  port(
11    Clk      : in  std_logic;
12    RstN     : in  std_logic;
13    Enable   : in  std_logic;
14    PixelIn  : in  std_logic_vector(7 downto 0);
15
16    Out00, Out01, Out02 : out std_logic_vector(7 downto 0);
17    Out10, Out11, Out12 : out std_logic_vector(7 downto 0);
18    Out20, Out21, Out22 : out std_logic_vector(7 downto 0)
19  );
20 end WindowGenerator;
21
22 architecture Structural of WindowGenerator is
23   type window_grid_type is array (0 to 2, 0 to 2) of std_logic_vector(7 downto 0);
24
25   signal WindowGrid : window_grid_type;
26   signal Buffer10Out : std_logic_vector(7 downto 0);
27   signal Buffer20Out : std_logic_vector(7 downto 0);
28 begin
29   -- NOTE(acol): grid to output
30   Out00 <= WindowGrid(0,0); Out01 <= WindowGrid(0,1); Out02 <= WindowGrid(0,2);
31   Out10 <= WindowGrid(1,0); Out11 <= WindowGrid(1,1); Out12 <= WindowGrid(1,2);

```

```

32 Out20 <= WindowGrid(2,0); Out21 <= WindowGrid(2,1); Out22 <= WindowGrid(2,2);
33
34 -- NOTE(acol): Instantiations
35 RingBuffer1: entity RingBuffer
36   generic map(
37     Depth => ImageWidth
38   )
39   port map(
40     Clk      => Clk,
41     RstN     => RstN,
42     Enable   => Enable,
43     -- NOTE(acol): Can make buffers 3 entries smaller this way
44     DataIn   => WindowGrid(2, 0),
45     DataOut  => Buffer1Out
46   );
47 RingBuffer2: entity RingBuffer
48   generic map(
49     Depth => ImageWidth
50   )
51   port map(
52     Clk      => Clk,
53     RstN     => RstN,
54     Enable   => Enable,
55     DataIn   => WindowGrid(1, 0),
56     DataOut  => Buffer2Out
57   );
58
59 process(Clk) begin
60   if rising_edge(Clk) then
61     if RstN = '0' then
62       -- NOTE(acol): zero out grid, will also feed 0s into fifos
63       for RowIdx in 0 to 2 loop
64         for ColIdx in 0 to 2 loop
65           WindowGrid(RowIdx, ColIdx) <= (others => '0');
66         end loop;
67       end loop;
68     elsif Enable = '1' then
69       -- NOTE(acol): shift grid left
70       for RowIdx in 0 to 2 loop
71         for ColIdx in 0 to 1 loop
72           WindowGrid(RowIdx, ColIdx) <= WindowGrid(RowIdx, ColIdx + 1);
73         end loop;
74       end loop;
75     end if;
76     -- NOTE(acol): get data from axi and fifos in image pixel order
77     WindowGrid(2, 2) <= PixelIn;
78     WindowGrid(1, 2) <= Buffer1Out;
79     WindowGrid(0, 2) <= Buffer2Out;
80   end if;
81 end if;
82 end process;
83 end Structural;
84
85

```

2. Debayering Filter

Για την δημιουργία του φίλτρου θα κρατάμε σε μετρητές τις συντεταγμένες του pixel εισόδου και του pixel που υπολογίζουμε. Από την θέση του pixel εισόδου ξέρουμε ποτέ έχουμε γεμίσει τις fifo, και άρα έχουμε πάρει αρκετά pixel για να αρχίσουμε να κάνουμε υπολογισμούς, και από την θέση του pixel εξόδου μπορούμε να ξέρουμε πότε βρισκόμαστε σε κάποια άκρη της εικόνας. Έκτος από το PipelineFilled θα κρατάμε και ένα ακόμα σήμα εσωτερικής κατάστασης, το IsProcessing, το οποίο χρησιμοποιούμε για να εφαρμόσουμε

την λογική γείρω από το σήμα NewImage. Το σύστημα θα περιμένει εκτός από τους κύκλους που παίρνει ValidIn και το NewImage ή το IsProcessing είναι high.

Επίσης για διευκόλυνση της λογικής στους υπολογισμούς θα δημιουργήσουμε με dataflow ένα πλέγμα το οποίο, ανάλογα του edge case, γεμίζουμε με μηδενικά ή τις τιμές των pixel που παίρνουμε από το 3×3 παράθυρο με statement της μορφής (others => '0') when TopEdge else unsigned(Out01);. Από αυτό θα υπολογίσουμε τα αθροίσματα για όλες τις περιπτώσεις, διαγώνια, κάθετα, οριζόντια και σε σταυρό, με κάποιο padding για αντιμετώπιση του overflow. Έτσι, όταν έρθει η ώρα να πάρουμε περιπτώσεις την τελική τιμή μπορούμε απλά να επιλέξουμε ένα από τα παραπάνω με ένα slide στα δεξιά.

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 use work.all;
5
6 entity GbrgDebayer is
7   generic(
8     ImageWidth : integer := 1024
9   );
10  port(
11    Clk      : in std_logic;
12    RstN     : in std_logic;
13    ValidIn  : in std_logic;
14    NewImage : in std_logic;
15    PixelIn  : in std_logic_vector(7 downto 0);
16
17    ValidOut   : out std_logic;
18    ImageFinished : out std_logic;
19    PixelR     : out std_logic_vector(7 downto 0);
20    PixelG     : out std_logic_vector(7 downto 0);
21    PixelB     : out std_logic_vector(7 downto 0)
22  );
23 end entity GbrgDebayer;
24
25 architecture Behavioral of GbrgDebayer is
26   constant ImageHeight : integer := ImageWidth;
27   type pixel_grid_type is array (0 to 2, 0 to 2) of unsigned(7 downto 0);
28
29   signal Out00, Out01, Out02 : std_logic_vector(7 downto 0);
30   signal Out10, Out11, Out12 : std_logic_vector(7 downto 0);
31   signal Out20, Out21, Out22 : std_logic_vector(7 downto 0);
32
33   signal PaddedGrid : pixel_grid_type;
34
35   signal InCol      : natural range 0 to ImageWidth - 1;
36   signal InRow      : natural range 0 to ImageHeight - 1;
37   signal CenterCol  : natural range 0 to ImageWidth - 1;
38   signal CenterRow  : natural range 0 to ImageHeight - 1;
39
40   -- NOTE(acol): internal state
41   signal PipelineFilled : std_logic;
42   signal IsProcessing   : std_logic;
43
44   -- NOTE(acol): helper variable
45   signal Enable : std_logic;
46
47   -- NOTE(acol): edge case flags
48   signal TopEdge, BottomEdge, LeftEdge, RightEdge : boolean;
49
50   -- NOTE(acol): will always calc them and just stuff the correct
51   -- thing to output after shifts
52   signal SumCross : unsigned(9 downto 0);
53   signal SumDiag  : unsigned(9 downto 0);
54   signal SumVert  : unsigned(8 downto 0);

```

```

55   signal SumHorz   : unsigned(8 downto 0);
56   begin
57
58   WindowGenInst: entity WindowGenerator
59     generic map(
60       ImageWidth => ImageWidth
61     )
62     port map(
63       Clk      => Clk,
64       RstN     => RstN,
65       Enable   => Enable,
66       PixelIn  => PixelIn,
67       Out00    => Out00, Out01 => Out01, Out02 => Out02,
68       Out10    => Out10, Out11 => Out11, Out12 => Out12,
69       Out20    => Out20, Out21 => Out21, Out22 => Out22
70     );
71
72   -- NOTE(acol): determine enable conditions
73   Enable <= ValidIn and (NewImage or IsProcessing);
74
75   -- NOTE(acol): Edge cases from output position
76   TopEdge    <= (CenterRow = 0);
77   BottomEdge <= (CenterRow = ImageHeight - 1);
78   LeftEdge   <= (CenterCol = 0);
79   RightEdge  <= (CenterCol = ImageWidth - 1);
80
81   -- NOTE(acol): mux logic to decouple edge cases from compute
82   -- we got A?B:C at home :)
83   PaddedGrid(0,0) <= (others => '0') when (TopEdge or LeftEdge) else unsigned(Out00);
84   PaddedGrid(0,1) <= (others => '0') when TopEdge else unsigned(Out01);
85   PaddedGrid(0,2) <= (others => '0') when (TopEdge or RightEdge) else unsigned(Out02);
86
87   PaddedGrid(1,0) <= (others => '0') when LeftEdge else unsigned(Out10);
88   PaddedGrid(1,1) <= unsigned(Out11);
89   PaddedGrid(1,2) <= (others => '0') when RightEdge else unsigned(Out12);
90
91   PaddedGrid(2,0) <= (others => '0') when (BottomEdge or LeftEdge) else unsigned(Out20);
92   PaddedGrid(2,1) <= (others => '0') when BottomEdge else unsigned(Out21);
93   PaddedGrid(2,2) <= (others => '0') when (BottomEdge or RightEdge) else unsigned(Out22);
94
95   -- NOTE(acol): compute common
96   SumCross <= ("00" & PaddedGrid(0,1)) + ("00" & PaddedGrid(1,0)) + ("00" & PaddedGrid(1,2))
+ ("00" & PaddedGrid(2,1));
97   SumDiag  <= ("00" & PaddedGrid(0,0)) + ("00" & PaddedGrid(0,2)) + ("00" & PaddedGrid(2,0))
+ ("00" & PaddedGrid(2,2));
98   SumVert  <= ('0' & PaddedGrid(0,1)) + ('0' & PaddedGrid(2,1));
99   SumHorz  <= ('0' & PaddedGrid(1,0)) + ('0' & PaddedGrid(1,2));
100
101   process(Clk, RstN) begin
102     if RstN = '0' then
103       InCol      <= 0;
104       InRow      <= 0;
105       CenterCol  <= 0;
106       CenterRow  <= 0;
107       PipelineFilled <= '0';
108       IsProcessing <= '0';
109       ValidOut    <= '0';
110       ImageFinished <= '0';
111       PixelR      <= (others => '0');
112       PixelG      <= (others => '0');
113       PixelB      <= (others => '0');
114
115     elsif rising_edge(Clk) then
116       if Enable = '1' then
117
118         if NewImage = '1' then
119           -- NOTE(acol): idle -> processing + reset state
120           IsProcessing <= '1';
121           InCol <= 1;
122           InRow <= 0;

```

```

123     CenterCol <= 0;
124     CenterRow <= 0;
125     PipelineFilled <= '0';
126     ValidOut <= '0';
127     ImageFinished <= '0';
128 else
129
130     -----
131     -- NOTE(acol): fifo handling
132     -----
133
134     -- NOTE(acol): keep track of input pixel position
135     if InCol /= ImageWidth - 1 then
136         -- if not in last col go next
137         InCol <= InCol + 1;
138     else
139         -- else wrap back
140         InCol <= 0;
141         if InRow /= ImageHeight - 1 then
142             -- if not last row go next
143             InRow <= InRow + 1;
144         else
145             -- else wrap back
146             InRow <= 0;
147         end if;
148     end if;
149
150     if PipelineFilled = '0' then
151         -- NOTE(acol): wait for enough inputs
152         if InRow = 1 and InCol = 1 then
153             PipelineFilled <= '1';
154             CenterRow <= 0;
155             CenterCol <= 0;
156         end if;
157     else
158         -- NOTE(acol): same logic as above but for the output pixel
159         if CenterCol /= ImageWidth - 1 then
160             CenterCol <= CenterCol + 1;
161         else
162             CenterCol <= 0;
163             if CenterRow /= ImageHeight - 1 then
164                 CenterRow <= CenterRow + 1;
165             else
166                 CenterRow <= 0;
167                 -- Stop when out of bounds
168                 PipelineFilled <= '0';
169             end if;
170         end if;
171     end if;
172
173     -- enable isnt needed here but keeping it for readability 🍷
174     ValidOut <= PipelineFilled and Enable;
175
176     -- NOTE(acol): success conditions
177     if PipelineFilled = '1' and CenterCol = ImageWidth - 1
178         and CenterRow = ImageHeight - 1 then
179         ImageFinished <= '1';
180         IsProcessing <= '0';
181     else
182         ImageFinished <= '0';
183     end if;
184
185     -----
186     -- NOTE(acol): Computations
187     -----
188
189     if PipelineFilled = '1' then
190         if (CenterRow mod 2 = 0) and (CenterCol mod 2 = 0) then
191             -- NOTE(acol): GB
192             PixelG <= std_logic_vector(PaddedGrid(1,1));

```

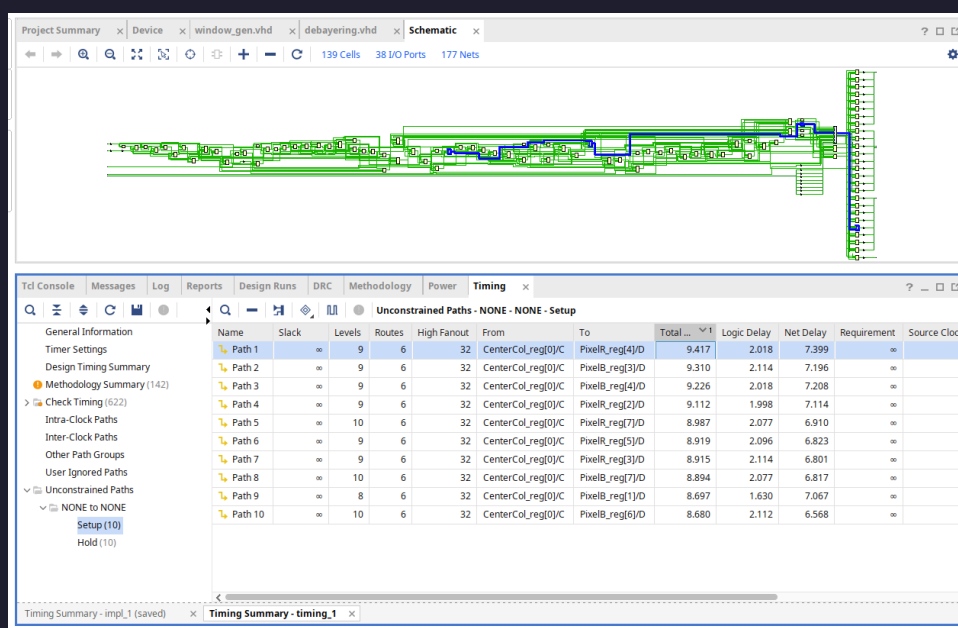
```

193         -- << 1
194         PixelB <= std_logic_vector(SumHorz(8 downto 1));
195         PixelR <= std_logic_vector(SumVert(8 downto 1));
196
197     elsif (CenterRow mod 2 = 0) and (CenterCol mod 2 /= 0) then
198         -- NOTE(acol): B
199         PixelB <= std_logic_vector(PaddedGrid(1,1));
200         -- << 2
201         PixelG <= std_logic_vector(SumCross(9 downto 2));
202         PixelR <= std_logic_vector(SumDiag(9 downto 2));
203
204     elsif (CenterRow mod 2 /= 0) and (CenterCol mod 2 = 0) then
205         -- NOTE(acol): R
206         PixelR <= std_logic_vector(PaddedGrid(1,1));
207         -- << 2
208         PixelG <= std_logic_vector(SumCross(9 downto 2));
209         PixelB <= std_logic_vector(SumDiag(9 downto 2));
210
211     else
212         -- NOTE(acol): GR
213         PixelG <= std_logic_vector(PaddedGrid(1,1));
214         -- << 1
215         PixelR <= std_logic_vector(SumHorz(8 downto 1));
216         PixelB <= std_logic_vector(SumVert(8 downto 1));
217     end if;
218 end if;
219 end if;
220
221 else
222     -- NOTE(acol): If not enable just wait
223     ValidOut <= '0';
224     ImageFinished <= '0';
225 end if;
226 end if;
227 end process;
228 end architecture;
229

```

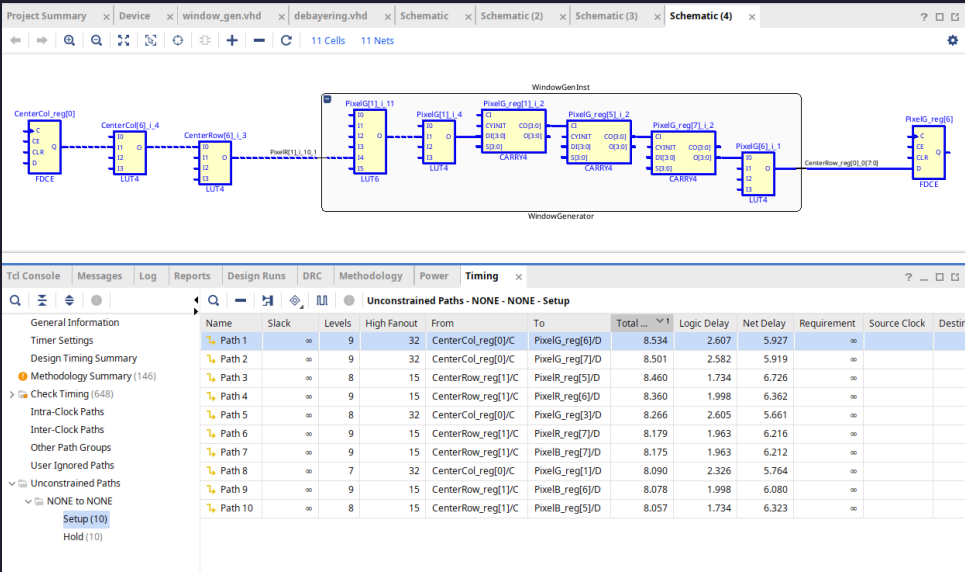
Παρακάτω είναι screenshot από implementation για critical path και utilization για εικόνες $N = 64, 128$ και 1024 pixel

2.1. 64



Name	^1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ N GbrgDebayer		163	138	60	163	1	38	1
▼ I WindowGenInst (WindowGenerator)		128	86	48	128	1	0	0
I RingBuffer1 (RingBuffer)		12	7	7	12	0.5	0	0
I RingBuffer2 (RingBuffer_0)		10	7	6	10	0.5	0	0

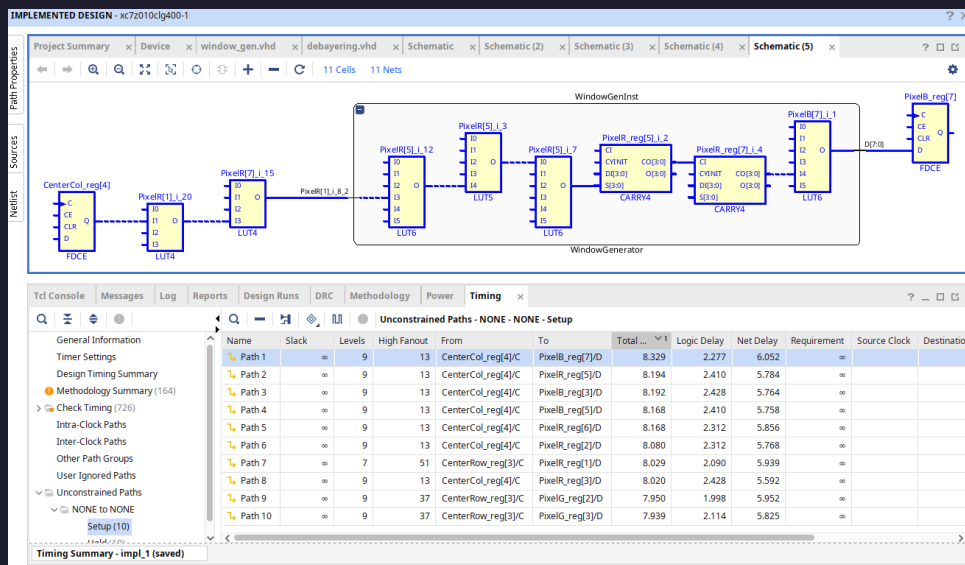
2.2. 128



Name	^1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ N GbrgDebayer		174	144	60	174	1	38	1
▼ I WindowGenInst (WindowGenerator)		132	88	48	132	1	0	0
I RingBuffer1 (RingBuffer)		13	8	7	13	0.5	0	0
I RingBuffer2 (RingBuffer_0)		11	8	7	11	0.5	0	0

Name	^1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ N GbrgDebayer		0.99%	0.41%	1.36%	0.99%	1.67%	38.00%	3.13%
▼ I WindowGenInst (WindowGenerator)		0.75%	0.25%	1.09%	0.75%	1.67%	0.00%	0.00%
I RingBuffer1 (RingBuffer)		0.07%	0.02%	0.16%	0.07%	0.83%	0.00%	0.00%
I RingBuffer2 (RingBuffer_0)		0.06%	0.02%	0.16%	0.06%	0.83%	0.00%	0.00%

2.3. 1024



Name	1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ GbgrDebayer		197	162	73	197	1	38	1
▼ WindowGenInst (WindowGenerator)		138	94	52	138	1	0	0
RingBuffer1 (RingBuffer)		15	11	7	15	0.5	0	0
RingBuffer2 (RingBuffer_0)		13	11	6	13	0.5	0	0

Name	1	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	Block RAM Tile (60)	Bonded IOB (100)	BUFGCTRL (32)
▼ GbgrDebayer		1.12%	0.46%	1.66%	1.12%	1.67%	38.00%	3.13%
▼ WindowGenInst (WindowGenerator)		0.78%	0.27%	1.18%	0.78%	1.67%	0.00%	0.00%
RingBuffer1 (RingBuffer)		0.09%	0.03%	0.16%	0.09%	0.83%	0.00%	0.00%
RingBuffer2 (RingBuffer_0)		0.07%	0.03%	0.14%	0.07%	0.83%	0.00%	0.00%